

ROBOT MARIPOSA CONTROLADO MEDIANTE ESP32 Y MPU6050 CON VISUALIZACIÓN EN UNITY

La Salle Bajío

Programacion Avanzada

Robot Mariposa Controlado mediante ESP32 y MPU6050 con Visualización en Unity

Integrantes:

- Samuel Hernández Oliva
- Josue Braulio Ravelo Lopez
- Nancy Cristal Largo Muñoz

Nombre del docente: Karla Daniela Martinez Esparza

Fecha de entrega: 17/06/2026

Resumen

El presente proyecto consiste en el diseño y desarrollo de una aplicación que simula el movimiento de una mariposa mediante la integración de hardware y software. Para lograrlo, se utilizó un microcontrolador ESP32 conectado a un sensor MPU6050, el cual permite detectar inclinaciones del dispositivo. Estos datos son procesados y transmitidos mediante comunicación Wi-Fi hacia una aplicación desarrollada en Unity, donde una mariposa tridimensional replica los movimientos detectados en tiempo real.

El objetivo principal del robot es simular el comportamiento de movimiento de un animal, en este caso una mariposa, utilizando sensores, programación orientada a objetos e interfaz gráfica. El sistema permite que, al inclinar físicamente el sensor hacia adelante, atrás, izquierda o derecha, la mariposa virtual se desplace en la misma dirección dentro del entorno de Unity.

La necesidad que busca resolver este proyecto es demostrar cómo se puede crear una interfaz visual que represente el comportamiento de un sistema físico mediante sensores y comunicación inalámbrica. Esto permite visualizar de manera clara los datos del hardware y convertirlos en movimiento dentro de una simulación gráfica.

El tipo de locomoción utilizada es una locomoción virtual basada en inclinación. No se implementaron motores físicos o servomotores en esta etapa, ya que el enfoque principal fue el control y visualización del movimiento mediante una mariposa 3D.

Las tecnologías empleadas fueron ESP32, sensor MPU6050, comunicación Wi-Fi, Arduino IDE, Unity, Visual Studio, C++ y C#. Entre las principales características del sistema se encuentran la transmisión inalámbrica de datos, la visualización en tiempo real, la animación de una mariposa virtual, la interfaz gráfica con indicadores y el uso de programación orientada a objetos.

Introducción

La robótica inspirada en animales permite estudiar y representar movimientos presentes en la naturaleza mediante sistemas electrónicos y computacionales. Este tipo de proyectos ayuda a comprender cómo se pueden combinar sensores, microcontroladores, programación y entornos gráficos para simular comportamientos biológicos.

En este proyecto se eligió el movimiento de una mariposa debido a que representa un sistema visualmente atractivo y permite trabajar con conceptos de inclinación, desplazamiento, animación y simulación tridimensional. En lugar de implementar inicialmente una estructura física compleja con alas y actuadores, se decidió comenzar con una simulación digital controlada por hardware real.

La motivación principal fue desarrollar un sistema que conectara el mundo físico con el mundo virtual. Para ello, se utilizó un MPU6050, que detecta inclinaciones del sensor, y un ESP32, que

transmite los datos por Wi-Fi hacia Unity. En Unity se creó una escena donde una mariposa animada responde a los movimientos físicos del sensor.

El alcance del sistema incluye la lectura de datos del sensor, el procesamiento de valores de Pitch y Roll, la comunicación inalámbrica mediante Wi-Fi, la recepción de datos en Unity, el movimiento de una mariposa virtual y la visualización de datos en una interfaz gráfica.

Entre las limitaciones identificadas se encuentran la precisión limitada del MPU6050, la dependencia de una red Wi-Fi local, posibles retardos en la comunicación inalámbrica y la ausencia de posicionamiento real mediante GPS. Aun así, el sistema cumple con el objetivo de simular el movimiento de una mariposa controlada mediante hardware real.

Descripción del Sistema

Componentes de Hardware

Microcontrolador utilizado

El microcontrolador utilizado fue un ESP32. Se eligió debido a que cuenta con conectividad Wi-Fi integrada, buena capacidad de procesamiento y compatibilidad con Arduino IDE.

Características principales del ESP32:

- Microcontrolador de bajo costo.
- Comunicación Wi-Fi integrada.
- Compatible con Arduino IDE.
- Alimentación mediante USB o fuente externa.
- Pines GPIO suficientes para conexión con sensores.
- Regulador de voltaje AMS1117 integrado.

Sensor implementado

El sensor utilizado fue el MPU6050, el cual contiene acelerómetro y giroscopio. Este sensor permite detectar la orientación del sistema mediante valores de inclinación.

Funciones principales del MPU6050:

- Lectura de aceleración en los ejes X, Y y Z.
- Lectura de velocidad angular.
- Cálculo de inclinación.
- Obtención de valores Pitch y Roll.
- Comunicación I2C con el ESP32.

Actuadores o servomotores

En esta etapa del proyecto no se implementaron actuadores físicos ni servomotores. El movimiento se representa de forma virtual dentro de Unity mediante una mariposa 3D animada. Sin embargo, el diseño puede ampliarse en el futuro para agregar servomotores que controlen alas físicas.

Sistema de alimentación

Durante las pruebas, el ESP32 fue alimentado mediante cable USB conectado a la computadora. Para una versión portátil, se propone utilizar un power bank de 5V, ya que el ESP32 cuenta con un regulador AMS1117 que permite alimentar la placa de forma segura.

Módulo de comunicación

La comunicación se realiza mediante Wi-Fi, utilizando la conectividad integrada del ESP32. El ESP32 se conecta a un hotspot de celular y Unity se conecta a la dirección IP del ESP32 para recibir los datos del sensor.

Componentes de Software

Lenguajes de programación utilizados

- C++ para la programación del ESP32 en Arduino IDE.
- C# para la programación de la aplicación en Unity.

Entornos de desarrollo

- Arduino IDE.
- Visual Studio.
- Unity 6.
- GitHub para almacenamiento del proyecto.

Librerías empleadas

En Arduino IDE:

- **Wire.h**: permite la comunicación I2C entre el ESP32 y el MPU6050.
- **MPU6050.h**: permite leer los datos del sensor MPU6050.
- **WiFi.h**: permite conectar el ESP32 a una red Wi-Fi.

En Unity:

- **UnityEngine**: permite controlar objetos, transformaciones y escena.

- **System.Net.Sockets**: permite la comunicación TCP/IP.
- **System.IO**: permite leer el flujo de datos recibido.
- **System.Threading**: permite leer datos en segundo plano.
- **System.Globalization**: permite interpretar correctamente valores decimales.
- **TMPPro**: permite mostrar textos en la interfaz gráfica.

Diagrama General de Bloques del Sistema

MPU6050
 ↓ Comunicación I2C
 ESP32
 ↓ Comunicación Wi-Fi
 Hotspot del celular
 ↓ Red local
 Computadora con Unity
 ↓
 Mariposa 3D animada

El MPU6050 detecta la inclinación del sensor. El ESP32 procesa los datos y obtiene los valores Pitch y Roll. Después, estos valores son enviados mediante Wi-Fi hacia Unity. Finalmente, Unity utiliza esos valores para mover, rotar y animar una mariposa virtual dentro de la escena.

Diagrama del Circuito

El circuito utilizado es simple, ya que únicamente se conecta el sensor MPU6050 al ESP32 mediante comunicación I2C.

Conexiones:

MPU6050	ESP32
VCC	3.3V
GND	GND
SDA	GPIO 21
SCL	GPIO 22

Descripción:

El pin VCC del MPU6050 se conecta a 3.3V del ESP32. El pin GND se conecta a tierra. Los pines SDA y SCL se conectan a los pines 21 y 22 del ESP32, respectivamente, ya que son los pines comúnmente utilizados para comunicación I2C en esta placa.

[Insertar aquí imagen del diagrama del circuito]

Explicación Visual de la Aplicación

La aplicación desarrollada en Unity muestra una mariposa 3D dentro de un escenario virtual. La mariposa responde a los movimientos del sensor físico conectado al ESP32.

Funcionamiento visual:

- Si el sensor se inclina hacia adelante, la mariposa avanza.
- Si el sensor se inclina hacia atrás, la mariposa retrocede.
- Si el sensor se inclina hacia la izquierda, la mariposa se desplaza hacia la izquierda.
- Si el sensor se inclina hacia la derecha, la mariposa se desplaza hacia la derecha.
- La animación de la mariposa permite simular el movimiento de sus alas.
- La interfaz muestra estado de conexión, IP del ESP32, Pitch y Roll.

[Insertar aquí captura de Unity con la mariposa funcionando]

Arquitectura del Programa

El programa se diseñó de manera modular para facilitar su mantenimiento, comprensión y escalabilidad. Aunque el proyecto se encuentra dividido entre Arduino y Unity, se puede interpretar como un sistema por capas.

Arquitectura por capas

Capa de presentación

Esta capa corresponde a la interfaz gráfica desarrollada en Unity. Su función es mostrar al usuario la simulación visual del sistema.

Elementos principales:

- Mariposa 3D.
- Escenario virtual.

- Cámara de seguimiento.
- Textos de estado.
- Valores Pitch y Roll.
- Estado de conexión Wi-Fi.

Capa de lógica

Esta capa interpreta los valores recibidos desde el ESP32. En Unity, los valores Pitch y Roll se convierten en movimiento y rotación para la mariposa.

Funciones:

- Determinar dirección de movimiento.
- Aplicar zona muerta para evitar vibraciones.
- Controlar velocidad.
- Mantener altura fija.
- Suavizar la rotación.

Capa de acceso a datos

Esta capa corresponde a la lectura del sensor MPU6050 desde el ESP32.

Funciones:

- Leer valores del acelerómetro.
- Calcular Pitch.
- Calcular Roll.
- Preparar los datos para enviarlos.

Capa de comunicación

Esta capa permite transmitir datos desde el ESP32 hacia Unity.

Funciones:

- Conectar ESP32 a Wi-Fi.
- Crear servidor TCP.
- Enviar datos Pitch y Roll.
- Recibir datos en Unity mediante TCP/IP.

Diseño con Clases

Clase del ESP32

En el ESP32, el programa se encarga de inicializar el sensor, conectarse a Wi-Fi y enviar datos. Aunque el código se encuentra en un archivo principal `main.ino`, conceptualmente se divide en módulos:

- Lectura del sensor.
- Procesamiento de inclinación.
- Comunicación Wi-Fi.

Clase SerialMPU en Unity

La clase `SerialMPU` controla la comunicación con el ESP32 y el movimiento de la mariposa.

Responsabilidades:

- Conectarse a la IP del ESP32.
- Recibir datos por TCP.
- Separar los valores Pitch y Roll.
- Actualizar la rotación de la mariposa.
- Actualizar la posición de la mariposa.
- Mostrar datos en la interfaz gráfica.

Clase de cámara

La cámara puede utilizar un script independiente para seguir a la mariposa durante la simulación.

Responsabilidades:

- Seguir la posición de la mariposa.
- Mantener una distancia visual.
- Apuntar hacia el objeto principal.

Principios SOLID Aplicados

SRP – Principio de Responsabilidad Única

Este principio indica que una clase debe tener una sola responsabilidad. En el proyecto se aplicó separando tareas principales:

- El ESP32 se encarga de leer el sensor y enviar datos.

- Unity se encarga de recibir datos y mostrar la simulación.
- La clase de movimiento se enfoca en controlar la mariposa.
- La interfaz gráfica se encarga de mostrar valores al usuario.

Esto facilita encontrar errores, modificar partes específicas y mantener el proyecto ordenado.

OCP – Principio Abierto/Cerrado

El sistema está diseñado para permitir agregar nuevas funcionalidades sin modificar completamente el código existente. Por ejemplo, en el futuro se pueden agregar:

- Nuevos sensores.
- Servomotores.
- Otro modelo 3D.
- Más indicadores en la interfaz.
- Otro tipo de comunicación.

El sistema se puede ampliar sin cambiar toda la estructura.

LSP – Principio de Sustitución de Liskov

Este principio se puede aplicar al considerar que el MPU6050 podría ser reemplazado por otro sensor de orientación, como un MPU9250 o un BNO055, siempre que entregue valores equivalentes de Pitch y Roll. Unity no necesita saber qué sensor se usa físicamente, solo recibe datos de orientación.

ISP – Principio de Segregación de Interfaces

El sistema evita concentrar todas las responsabilidades en una sola parte. En lugar de una clase gigante que haga todo, se separan funciones como lectura del sensor, comunicación, movimiento y visualización. Esto ayuda a evitar módulos excesivamente grandes.

DIP – Principio de Inversión de Dependencias

Unity no depende directamente del sensor MPU6050, sino de los datos recibidos mediante Wi-Fi. Esto significa que la aplicación puede funcionar con cualquier hardware que envíe datos en el formato correcto.

Ejemplo de formato:

Pitch,Roll

Ejemplo real:

12.35,-4.28

Comunicación Computador – Microcontrolador

La comunicación entre la computadora y el microcontrolador se realiza mediante Wi-Fi. El ESP32 se conecta a un hotspot de celular y crea un servidor TCP en el puerto 5000.

Unity actúa como cliente TCP y se conecta a la dirección IP del ESP32. Una vez establecida la conexión, el ESP32 envía continuamente los valores de Pitch y Roll.

Tecnología utilizada

- Wi-Fi.
- TCP/IP.
- ESP32 como servidor.
- Unity como cliente.
- Hotspot de celular como red local.

Datos transmitidos

Los datos enviados son:

- Pitch: inclinación hacia adelante o atrás.
- Roll: inclinación hacia izquierda o derecha.

Formato de transmisión:

Pitch,Roll

Ejemplo:

8.25,-3.40

Ventajas de usar Wi-Fi

- El sistema no depende del cable USB para funcionar.
- Permite mayor libertad de movimiento.
- Aprovecha la conectividad integrada del ESP32.
- Hace que el proyecto sea más presentable y cercano a un sistema real inalámbrico.

Interfaz Gráfica de Usuario

La interfaz gráfica fue desarrollada en Unity. Su objetivo es permitir al usuario visualizar el comportamiento de la mariposa y monitorear los datos del sistema.

Características principales

Ventana principal

Muestra el escenario virtual donde se encuentra la mariposa 3D.

Indicadores de estado

La interfaz muestra si Unity se encuentra conectado o desconectado del ESP32.

Ejemplo:

Estado: Conectado

Monitoreo de sensores

Se muestran los valores recibidos desde el ESP32:

Pitch: 12.35°

Roll: -4.28°

Dirección IP

La interfaz muestra la IP del ESP32 para verificar que Unity se está conectando al dispositivo correcto.

Ejemplo:

IP: 10.227.41.120

Cámara de seguimiento

La cámara sigue a la mariposa para que el usuario pueda observar su movimiento dentro de la escena.

Animación

La mariposa utiliza una animación de vuelo, permitiendo que el movimiento visual sea más parecido al comportamiento real de una mariposa.

[Insertar captura de la interfaz gráfica]

Funcionamiento General del Código

Código en ESP32

El código del ESP32 realiza las siguientes acciones:

1. Inicializa la comunicación serial para depuración.
2. Inicializa el sensor MPU6050 mediante I2C.
3. Se conecta a una red Wi-Fi.
4. Muestra la IP del ESP32 en el Monitor Serial.
5. Crea un servidor TCP en el puerto 5000.
6. Lee datos del MPU6050.
7. Calcula Pitch y Roll.
8. Envía los datos a Unity.

Código en Unity

El código en Unity realiza las siguientes acciones:

1. Se conecta a la IP del ESP32.
2. Abre una conexión TCP.
3. Recibe datos continuamente.
4. Separa los valores Pitch y Roll.
5. Aplica zona muerta para evitar vibraciones.
6. Calcula la dirección de movimiento.
7. Mueve la mariposa dentro del escenario.
8. Mantiene la altura fija.
9. Actualiza la interfaz gráfica.

Implementación del Diseño del Proyecto

El diseño se implementó primero en protoboard para validar el funcionamiento del sensor y del ESP32. La primera prueba consistió en verificar que el MPU6050 fuera detectado correctamente mediante la dirección I2C 0x68.

Después se realizaron pruebas de lectura del acelerómetro y se obtuvieron valores AX, AY y AZ. Posteriormente se calcularon los valores Pitch y Roll para interpretar la inclinación del sensor.

Una vez que el hardware funcionó correctamente, se creó una escena en Unity con un objeto 3D. Primero se probó el movimiento con una cápsula y después se reemplazó por un modelo de mariposa. Se corrigieron los ejes de movimiento para que la dirección física del sensor coincidiera con la dirección visual de la mariposa.

Finalmente, se migró la comunicación de USB Serial a Wi-Fi. El ESP32 se conectó a un hotspot de celular y Unity recibió los datos mediante TCP/IP.

Resultados

Durante las pruebas se logró que la mariposa virtual respondiera a los movimientos físicos del sensor MPU6050.

Experimentos realizados

Prueba 1: Detección del MPU6050

Se utilizó un escáner I2C para verificar que el sensor fuera detectado. El resultado fue exitoso al encontrar el dispositivo en la dirección 0x68.

Prueba 2: Lectura de aceleraciones

Se obtuvieron valores de aceleración en los ejes X, Y y Z. Al mover el sensor, los valores cambiaban correctamente.

Prueba 3: Movimiento en Unity por USB

Se logró que un objeto en Unity respondiera a los datos enviados por el ESP32 mediante comunicación serial USB.

Prueba 4: Corrección de ejes

Se realizaron ajustes para que la inclinación física del sensor coincidiera con el movimiento de la mariposa virtual.

Prueba 5: Comunicación Wi-Fi

Se configuró el ESP32 para conectarse a un hotspot de celular y enviar datos a Unity mediante TCP/IP. La prueba fue exitosa.

Prueba 6: Interfaz gráfica

Se agregaron textos en Unity para mostrar estado de conexión, IP, Pitch y Roll.

Problemas Encontrados y Soluciones

Problema 1: El ESP32 no cargaba código

Se presentó un error con el puerto COM. La solución fue instalar el driver CP2102 correspondiente y seleccionar el puerto correcto en Arduino IDE.

Problema 2: El MPU6050 no respondía

Se verificó la conexión I2C y se confirmó que el sensor aparecía en la dirección 0x68.

Problema 3: El cubo en Unity se movía de forma caótica

Se ajustaron los cálculos de Pitch y Roll y se aplicaron zonas muertas para evitar vibraciones.

Problema 4: La mariposa se movía en dirección contraria

Se invirtieron los ejes correspondientes en el código de Unity para que el movimiento físico coincidiera con la simulación.

Problema 5: La comunicación Wi-Fi no conectaba

Se verificó que la computadora y el ESP32 estuvieran conectados al mismo hotspot. También se actualizó la IP del ESP32 en Unity.

Problema 6: La animación movía el modelo hacia abajo

Se revisó la configuración del Animator y se desactivó el movimiento raíz cuando fue necesario.

